

Día, Fecha:

Viernes, 27/02/2026

Hora de inicio:

19:05

Inteligencia Artificial 1 [A]

Roberto Miguel García Santizo

ROBERTO GARCÍA

INTELIGENCIA ARTIFICIAL 1

CLASE 04

AGENDA

- Preguntas sobre el proyecto
- Dudas sobre el corto
- Asistencia
- Clase teorica
- Continuacion del ejercicio

ASISTENCIA



COMPETENCIAS A DESARROLLAR

Desarrollar hechos, reglas,
expresiones y predicados
recursivos mediante el uso
de cláusulas, ciclos,
listas, unificación y
cortes en Prolog para
modelar bases de
conocimiento y resolver
problemas de inferencia
lógica

CORTES



El corte en prolog se representa con el simbolo:
“!”

Se utiliza para controlar el backtracking (retroceso) y hacer que Prolog no busque más soluciones después de cierto punto.

CORTES

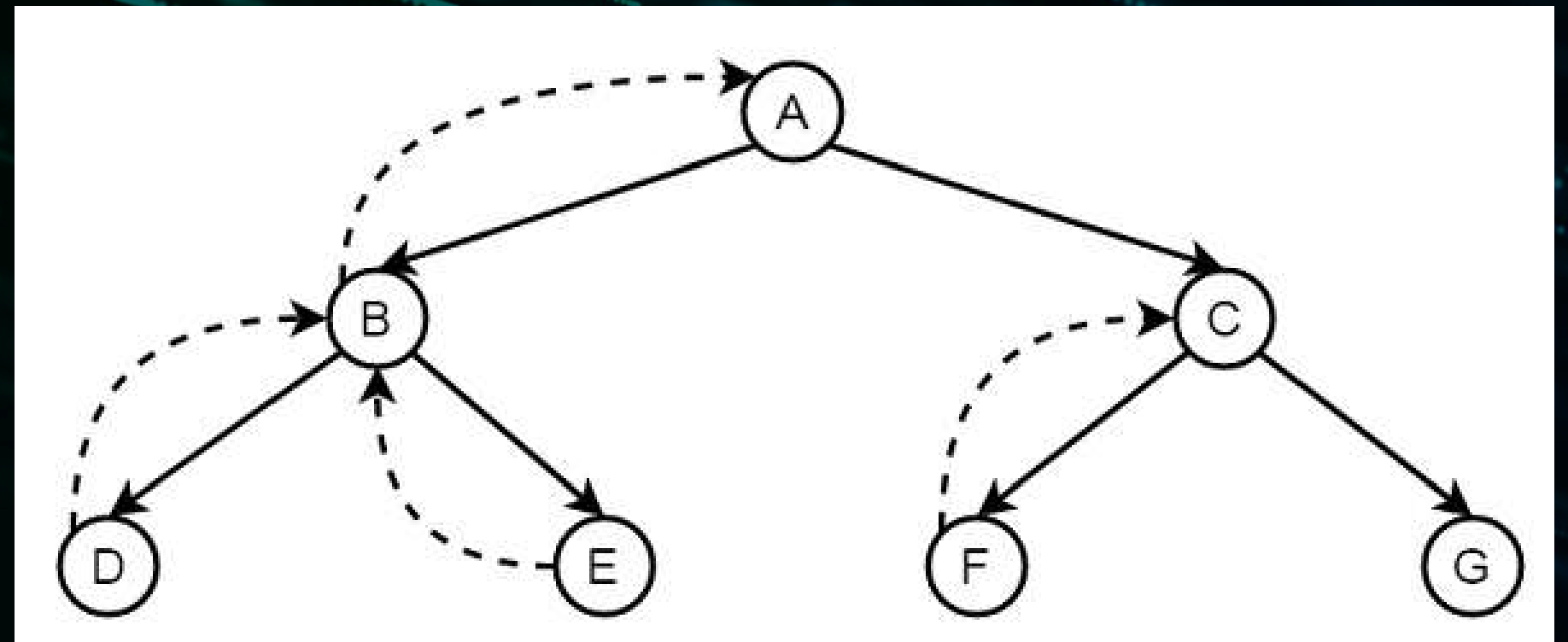
El corte es un predicado especial que:

- Siempre tiene éxito
- pero elimina alternativas anteriores
- Evita que Prolog regrese atrás

BACKTRACKING

Prolog busca soluciones:

- Intenta una regla
- Si falla, regresa y prueba otra
- Sigue probando hasta encontrar todas las soluciones posibles



CARACTERÍSTICAS

- Se escribe como !
- Siempre es verdadero
- No genera valores
- Controla el flujo del programa
- Mejora eficiencia (evita cálculos innecesarios)
- Puede cambiar la lógica si se usa mal

EJEMPLO SIN CORTE



```
1 mayor(X, Y, X) :- X > Y.  
2 mayor(X, Y, Y) :- X <= Y.
```

```
59 ?- mayor(5,3,R).
```

```
R = 5 ;
```

```
false.
```

```
60 ?- 
```

Aqui Prolog prueba la primera regla y funciona

EJEMPLO CON CORTE



```
1 mayor(X, Y, X) :- X > Y, !.  
2 mayor(X, Y, Y).
```

```
61 ?- mayor(5,3,R).  
R = 5.
```

```
62 ?- mayor(1,3,R).  
R = 3.
```

```
63 ?- 
```

¿Qué hace el “!”?

Si $X > Y$ es verdadero:

- Ejecuta el corte
- No intenta la segunda regla
- Evita seguir buscando

TIPOS DE CORTE

Corte Verde (Green Cut)

- No cambia el resultado lógico
- Solo mejora eficiencia



```
1 max(X,Y,X) :- X >= Y, !.  
2 max(_,Y,Y).
```

El resultado es el mismo con o sin corte.
Solo evita trabajo innecesario

Corte rojo (Red Cut)

- Cambia el comportamiento lógico.
- Puede eliminar soluciones correctas.



```
1 valor(1) :- !.  
2 valor(2).  
3  
4 # consulta  
5 ?- valor(X).  
6  
7 # Respuesta  
8 X = 1
```

Nunca devuelve X = 2 porque el
corte bloquea la segunda regla.

EJEMPLO



```
1 clasificar(X, positivo) :- X > 0, !.  
2 clasificar(0, cero) :- !.  
3 clasificar(_, negativo).
```

For online help and background, visit <https://www.swi-prolog.org>
For built-in help, use `?- help(Topic).` or `?- apropos(Word).`

```
63 ?- clasificar(5,R).  
R = positivo.
```

```
64 ?- clasificar(0,R).  
R = cero.
```

```
65 ?- clasificar(-12.5,R).  
R = negativo.
```

```
66 ?- clasificar(1,R).  
R = positivo.
```

```
67 ?- 
```

RESUMEN

Ventajas del corte

- Mejora redimiento
- Controla el flujo
- Evita soluciones duplicadas
- Permite programación determinista

Desventajas del corte

- Puede eliminar soluciones válidas
- Hace el código menos declarativo
- Puede generar errores difíciles de detectar

¿Cuándo usar corte?

- Cuando solo se necesita una solución
- Cuando se sabe que no debe de existir alternativas
- Optimizar reglas condicionales

Comparación rápida

- Sin corte:
 - Prolog busca todas las soluciones
- Con corte:
 - Prolog se detiene cuando encuentra la primera válida

NEGACIÓN

En Prolog la negación se
representa como: “\+”

Prolog usa Negación por
Falla (Negation as
Failure).

NEGACIÓN

Sintaxis:
“\+ predicado”

Prolog intenta demostrar el predicado:

- Si lo logra $\rightarrow$ la negación falla.
- Si no puede demostrarlo $\rightarrow$ la negación es verdadera.

EJEMPLO #1



```
1 hombre(juan).  
2 hombre(pedro).  
3
```

For online help and background
For built-in help, use `?-`

```
90 ?- hombre(juan).  
true.
```

```
91 ?- \+ hombre(juan).  
false.
```

```
92 ?- 
```

EJEMPLO #2



```
1  ciudadano(ana).  
2  ciudadano(luis).  
3  
4  menor_edad(luis).  
5  
6  puede_votar(X) :- ciudadano(X), \+ menor_edad(X).
```

```
PS C:\Programacion\IA1\ejemplos-proprios\ejemplos> swipl .\ejem.pl  
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.0)  
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.  
Please run ?- license. for legal details.
```

```
For online help and background, visit https://www.swi-prolog.org  
For built-in help, use ?- help(Topic). or ?- apropos(word).
```

```
72 ?- puede_votar(ana).  
true.
```

```
73 ?- puede_votar(luis).  
false.
```

```
74 ?- 
```

RESUMEN NEGACIÓN

Ventajas

- Permite condiciones tipo “si no...”
- Útil en reglas lógicas
- Fácil de usar

Desventajas

- No es negación lógica real
- Puede fallar con variables no instanciadas
- Puede producir resultados engañosos

RECURSIÓN

La recursión ocurre cuando un predicado se llama a sí mismo para resolver un problema más pequeño.

Es fundamental en Prolog porque:

- No se usan ciclos como for o while
- Las repeticiones se hacen con recursión

RECURSIÓN

Es una técnica donde una regla se define en términos de sí misma.

Siempre debe tener:

- Caso base (condición que detiene la recursión)
- Caso recursivo (la llamada a sí mismo)

ESTRUCTURA GENERAL



```
1 predicado(CasoBase).  
2  
3 predicado(ProblemaMayor) :-  
4     condicion,  
5     predicado(ProblemaMasPequeño).
```

TIPOS DE RECURSIÓN

Recursión Simple (Lineal)

El predicado se llama una sola vez dentro de su definición.

Es el mas común

For online help and background, visit <https://www.swi-prolog.org>
For built-in help, use `?- help(Topic).` or `?- apropos(Word).`

```
75 ?- factorial(4,R).  
R = 24 ;  
false.
```

```
76 ?-  
```



```
1 factorial(0,1).  
2  
3 factorial(N,R) :-  
4     N > 0,  
5     N1 is N - 1,  
6     factorial(N1,R1),  
7     R is N * R1.
```

TIPOS DE RECURSIÓN

Recursión de Cola (Tail Recursion)
La llamada recursiva es la última
operación que se ejecuta.

Es más eficiente.

```
77 ?- suma(4,0,R).  
R = 10 .
```

```
78 ?- suma(0,0,R).  
R = 0 .
```

```
79 ?- suma(1,0,R).  
R = 1 ;  
false.
```

```
80 ?- suma(2,0,R).  
R = 3 ;  
false.
```

```
81 ?- 
```



```
1 suma(0,Acc,Acc).  
2  
3 suma(N,Acc,R) :-  
4     N > 0,  
5     N1 is N - 1,  
6     Acc1 is Acc + N,  
7     suma(N1,Acc1,R).
```

TIPOS DE RECURSIÓN

Recursión Múltiple

El predicado se llama más de una vez dentro de la misma regla.

For online help and background information, see
For built-in help, use `help`.

```
82 ?- fib(5,R).
```

```
R = 5 .
```

```
83 ?- 
```



```
1 fib(0,0).  
2 fib(1,1).  
3  
4 fib(N,R) :-  
5     N > 1,  
6     N1 is N - 1,  
7     N2 is N - 2,  
8     fib(N1,R1),  
9     fib(N2,R2),  
10    R is R1 + R2.
```

TIPOS DE RECURSIÓN

Recursión Múltiple

El predicado se llama más de una vez dentro de la misma regla.

For online help and back
For built-in help, use

```
82 ?- fib(5,R).
```

```
R = 5 .
```

```
83 ?- 
```



```
1  fib(0,0).  
2  fib(1,1).  
3  
4  fib(N,R) :-  
5      N > 1,  
6      N1 is N - 1,  
7      N2 is N - 2,  
8      fib(N1,R1),  
9      fib(N2,R2),  
10     R is R1 + R2.
```

TIPOS DE RECURSIÓN

Recursión Mutua

Dos predicados se llaman entre sí.

For online help and built-in help, use `help` and `help2`.

```
84 ?- par(4).  
true .
```

```
85 ?- par(3).  
false.
```

```
86 ?- par(0).  
true .
```

```
87 ?- 
```



```
1 par(0).  
2  
3 par(N) :-  
4     N > 0,  
5     N1 is N - 1,  
6     impar(N1).  
7  
8 impar(N) :-  
9     N > 0,  
10    N1 is N - 1,  
11    par(N1).
```

ERRORES COMUNES

No definir caso base
No reducir el problema
Orden incorrecto de condiciones
Recursión infinita



```
1 malo(N) :-  
2     malo(N).
```

COMPARACIÓN DE TIPO

Tipo	Llamadas	Eficiencia
Simple	1	Normal
Cola	1 (Ultima operación)	Alta
Multiple	2 o más	Baja
Mutua	Entre predicados	Variable
Listas	1 sobre cola	Normal

HABILIDADES DESARROLLADAS

- Analizar el flujo de ejecución en Prolog.
- Implementar cortes verdes y rojos correctamente.
- Diseñar predicados recursivos con caso base.
- Evaluar cuándo usar negación por falla.

CONCEPTOS CLAVE APRENDIDOS

- Corte en Prolog: El corte (!) controla el retroceso en la ejecución. El corte verde mejora la eficiencia sin cambiar la lógica. El corte rojo modifica la lógica del programa y debe usarse con precaución.
- Negación como Falla: Prolog usa \+ para negar: algo es falso si no se puede probar como verdadero. No es lógica clásica y tiene limitaciones con variables no instanciadas.
- **Recursión:** Prolog utiliza recursión en lugar de bucles. La recursión directa es más simple, pero la de cola es más eficiente y evita problemas de memoria.
- **Paradigma Declarativo y Simulación:** Prolog describe lo que se quiere lograr, no cómo. Es útil en IA y simulaciones por su enfoque lógico y expresivo.

Comprender estos mecanismos permite escribir programas más eficientes, claros y deterministas

PERSEVERANCIA

La perseverancia implica mantener el esfuerzo constante frente a la dificultad, insistiendo en encontrar soluciones y no rendirse fácilmente ante los errores o la complejidad de un problema.

DUDAS?



CONTINUACIÓN DEL EJERCICIO

```
        'role_id' => $role_details['id'],  
        'resource_id' => $resource_details['id'],  
    );  
    if ( $this->rule_exists( $resource_details['id'], $role_details['id'] ) )  
    {  
        if ( $access == false )  
        {  
            // Remove the rule as there is currently no need for it  
            $details['access'] = !$access;  
            $this->_sql->delete( 'acl_rules', $details );  
        } else {  
            // Update the rule with the new access value  
            $this->_sql->update( 'acl_rules', array( 'access' => $access ) );  
        }  
    }  
    foreach( $this->rules as $key=>$rule ) {  
        if ( $details['role_id'] == $rule['role_id'] && $details['resource_id'] == $rule['resource_id'] )  
        {  
            if ( $access == false )  
            {  
                unset( $this->rules[ $key ] );  
            } else {  
                $this->rules[ $key ]['access'] = $access;  
            }  
        }  
    }  
}
```

THANK YOU

...

...

